BridgeClaims

Smart contract audit / five-chain bridge-collateral claims / final report

29 October 2026

REVIEWED SOURCE	BridgeClaims.sol / 287 lines / 13,575 bytes
SHA-256	`94f8bed2e34d5a55a2304e96ab256cf59e980f5c813271c5659fd397c0f71255`

All High and Medium findings closed. Signing is next.

The contract is unchanged, so every contract-level result carries forward. The tooling now fails closed, and run 20261029's five sealed packages decode to the right spender, amounts and nonces.

0 CRITICAL	0 HIGH	0 MEDIUM	2 LOW	3 INFO

STATUS AT 06a3f82

CLOSED **2 High, 3 Medium and 6 Low findings remediated**

Spender bound to bytecode; contract payees held; direct factory deploy; sealed, locked packages.

L04 **ONE stage 1 still to execute; one new Low (L09)**

Rebuilt at Reserve nonce 61, still live at the final read. L09: a deleted lock file fails closed.

INFO **Three informational follow-ups**

Dust-row policy before opening (I09); hygiene gate red on PDFs (I05); key custody (I10).

The crosswalk maps both series' IDs onto the team's IDs; BC-I11 and BC-I12 are closed, confirmed by ten adversarial probes.

INDEPENDENT VALIDATION

Reproduced at 06a3f82. Test gate at 06a3f82: 30 Forge, 57 tool and 14 allocation tests pass. Both original PoCs re-run and stop before writing. Public (22 objects) and private (530 files) indexes verified; five seals verified; batches decoded and Safe hashes recomputed; runtime hashes rebuilt.

Carried forward unchanged: invariants I1 to I7, 836,756-row proof verification, 100% coverage, the 131,072-call invariant campaign and the 1,366,030-case differential Merkle test, because the source hash is unchanged.

BUILD & RELEASE STATUS

Solidity 0.8.36 / Cancun / optimizer 20,000 / no metadata; creation keccak 0x63cba289...0cb9. Run 20261029: 836,754 rows, four approval packages at Safe nonce 1 and one ONE stage-1 package at Reserve nonce 61, all current. Custody Safe: four roster owners, threshold 2; no code at any claim address.

Scope: BridgeClaims.sol, the deployment script, the list builder and proof tools, approval, package, quarantine and release generation, the custody Safe on Ethereum, BSC, Arbitrum, Base and Linea, and run bridge-allocations-20261029, at repository commit 06a3f82. Consolidates the audit of 4840fa9 and the remediation reviews through the 5a8937e regrade.

Mythos 5.1, Fable 5.1 Max, GPT-6 Astra Ultra, Daybreak Blue, and Opus 5.5 were used for preparing and conducting this audit. hiddenstate.xyz participates in OpenAI's Trusted Access for Cyber (TAC) and Anthropic's Cyber Verification Program.

Executive summary

This is the final audit report for BridgeClaims at repository commit `06a3f82`. It consolidates the audit of commit `4840fa9` (the Mythos pass and the Daybreak first pass, 28 October 2026), the Daybreak remediation reviews of `91f408b`, `284dc77` and `5a8937e` and the regrade of `5a8937e` (28 and 29 October 2026), and the Mythos verification of the remediation at `06a3f82` (29 October 2026), which covers the commits that landed after the last Daybreak pass. It has one finding list, one ID scheme (the team's, from `bridge-claims/docs/AUDIT_RESPONSE.md`) and one decision. The evidence base is the Mythos audit; Daybreak evidence is cited where it verifies a remediation or a publication, and the team's responses are quoted by commit.

At `06a3f82` every High and Medium finding of both series is remediated and verified. The contract source is byte-identical to the audited source, so every contract-level result of the original audit carries forward unchanged. The approval tool now recomputes the spender from the compiled bytecode and writes signable files only when every check at one pinned block passes; packages are sealed, listed in a run record, written under one run lock and quarantined with a journal; the release planner derives its recipient from the configuration, binds the release to the claim run and to the published index, and reads each chain at one pinned block; the deployment script calls the canonical factory directly and binds the init code, the address and the runtime code. The verification pass re-ran both original proofs of concept against the new tool, and both stop before anything signable is written.

The canonical claim run `bridge-allocations-20261029` has 836,754 rows, two fewer than the audited run because the two Ethereum rows payable to Harmony contracts are now held. Its public index (22 objects) and the private index (530 files) verified by hash in the verification pass, every local run file matches the public index, and its five sealed packages (four chain approvals at custody Safe nonce 1, one ONE stage-1 transfer at Reserve Safe nonce 61) decode to the right spender, amounts and recipient, with Safe transaction hashes that the verification pass recomputed independently. At one pinned block per chain the custody Safe has the four roster owners and threshold 2, no claim address has code, and the Reserve Safe nonce is still 61.

Open at `06a3f82`: 0 Critical, 0 High, 0 Medium, 2 Low, and 3 Informational follow-ups. The Lows are BC-L04, the ONE stage-1 transfer, which is rebuilt and verified but not yet executed, and BC-L09, a new finding of the verification pass: the run lock is tied to a deletable file, so a lock file removed during a refresh lets a second refresh in, after which the first process's failure handler invalidates the second's good package. Every interleaving observed fails closed and nothing signable results. The follow-ups are the dust-row relayer policy (BC-I09), the repository hygiene gate, which is red on tracked PDFs (BC-I05), and owner key-custody documentation (BC-I10). The Daybreak regrade's hardening items BC-I11 and BC-I12 are closed by the run lock, run record, journaled quarantine and run binding of `c8ced27`, which an adversarial review could not break in ten probes and 104 checks.

Where the two series rated the same defect differently, the merged rating follows the plan's rubric: BC-H01 Medium, BC-H02 Low, BC-M01 Low, BC-M02 Medium, BC-M03 Low at `4840fa9`. The team's IDs are kept as labels; the crosswalk below records both original ratings and the reason for each merged rating. No issue is counted twice.

Per-chain decision at `06a3f82`, separate from finding status:

- ONE stage 1 (Ethereum, Reserve Safe): conditional GO. The package is current, sealed, bound to the run and built at nonce 61, which was still the live nonce at the verification pass's pinned read; rebuild it if the nonce moves, and have the signers compare the Safe transaction hash with the sheet.
- Sign approvals: conditional GO on BSC, Arbitrum, Base and Linea; NO on Ethereum until ONE stage 1 has executed and an Ethereum approval package is built. The conditions are those of the runbook: `tools/package.py verify` reports the package current, each owner has confirmed their roster address, the Safe nonce is unchanged or the package is refreshed, and a second party reproduces every call and hash.
- Deploy: conditional GO on all five chains, from the same run, with a fresh simulation on the target chain immediately before broadcasting and the post-deployment runtime and immutable check afterwards.

- Open claims: NO on every chain until the deployed runtime hash and all six immutables are verified on the real nodes, allowances equal the approvals, random and negative proofs are exercised through the real RPC, per-asset canaries pass with a documented revoke process, and the dust-row policy is decided. Ethereum additionally needs ONE stage 1 executed.

Audit identity

Item	Value
Contract	`bridge-claims/src/BridgeClaims.sol` (unchanged since `440ec9b`)
Source lines / bytes	287 / 13,575
Source SHA-256	`94f8bed2e34d5a55a2304e96ab256cf59e980f5c813271c5659fd397c0f71255`
Audit plan SHA-256	`751141d67095f509d23dda806c6116eacbf6565dd13e0ca0230b5b7403038745`
Team response SHA-256 (at `06a3f82`)	`c30872e9116f0de597652ffb17612c2af25de0d215137807e5e9d92a12d30706`
Custody roster SHA-256	`7b958344e93d9b9286aa7d5b57859f8f41752d634a4c8a96c01fb8243eddd31e`
Compiler	Solidity `0.8.36+commit.8a079791`
Settings	Cancun, optimizer enabled, 20,000 runs, legacy codegen, no CBOR metadata
Creation code	5,626 bytes; Keccak `0x63cba28992049e747767818863cd42803dc10e1d8ad093d99e3725db5ffd0cb9`
Runtime template	5,120 bytes; Keccak `0xb36b3fff0e95016effd735265a52c6baaffcea66e90b0485e64c4084aec07cd3`
Public index `SHA256SUMS` (20261029)	`718c44d94c5105b5e36603c226ce46f1231d6368d96b2ac59c573d8a72cc46fb`
Private index `MANIFEST-20261029c-bridge-claims.tsv`	`acfc2b8e865b3c9ad35cc9cdaa7f133dac40a819bbb56648f817c7fc95761cb2`
Run record `PACKAGES.json` (revision 6)	`5d67813b6c8ba409c71149fe7a01a9037236a84b9f6605173ae20d68fdb30fc2`

The tooling changed between `4840fa9` and `06a3f82` in fourteen files under `bridge-claims/tools`, `script`, `chains.json`, `custody-roster.json` and `toolkit/scripts/bridge/build-bridge-allocations.py` (2,461 insertions, 269 deletions); the tools at `06a3f82` are hash-pinned in `manifests/source-code.sha256`. The contract, `foundry.toml`, `tools/build_claims.py`'s list logic, `tools/claim_tree.py`'s tree logic, `airdrop/tools/common.py` and `safe_batch.py` are unchanged in what they compute.

Scope and threat model

The scope is the plan's: the contract, its compiler and deployment configuration, the CREATE2 deployment script, the Merkle list builder and proof consumers, approval and revocation generation, the custody Safe's identity on five chains, escrow-release planning as a prerequisite integration, the payee claim flow, and the artifact set. This report applies it to the canonical run `bridge-allocations-20261029` and to the tooling at `06a3f82`, and treats the superseded runs `20261018` and `20261028` as quarantined evidence.

Assets sit in the 2-of-4 custody Safe `0x19565f4771843467aad632d6b56c75396785b06c`, the same address on all five chains, and are exposed only through per-token allowances to one immutable contract per chain. Anyone can submit a valid claim, so a claimant must not be able to redirect value. The deployer is untrusted. Token issuers, the canonical wrapped-native contracts, the Safe implementation and the LayerZero escrows remain external dependencies. Address-level claim data, Safe-owner identities, key-custody details and private RPC information are not reproduced.

Inputs and method

Input	Commit	Date	What it establishes
Daybreak first audit	`4840fa9`	28 Oct	First-pass findings BC-H01, BC-H02, BC-M01, BC-M02, BC-M03, BC-L01, BC-I01, BC-I02
Mythos second-pass audit	`4840fa9`	28 Oct	Full plan execution with twelve phases and six independent verifications; re-grades of the first pass; BC-L04 to BC-L08 and BC-I03 to BC-I10
Daybreak remediation review	`91f408b`	28 Oct	BC-H01, BC-H02, BC-M01 verified remediated; independent rebuild of the Ethereum list (187,005 rows); BC-M02 and BC-M03 partly open
Daybreak review	`284dc77`	28 Oct	Public bundle `20261028` verified (22 objects); seal and activation counterexamples for BC-M02; summary-to-manifest counterexample for BC-M03
Daybreak review	`5a8937e`	28 Oct	Public bundle `20261029` verified with every proof; BC-L05, BC-L06, BC-L07 verified in code or artifact; lock-scope and quarantine counterexamples
Daybreak regrade	`5a8937e`	29 Oct	BC-M02 and BC-M03 verified remediated; residuals moved to BC-I11 and BC-I12
Mythos verification pass	`06a3f82`	29 Oct	Commits `87380e0` to `06a3f82`: committed roster, run lock and record, `package.py verify`, journaled quarantine, `claims_run.py` binding; test gate; PoC re-runs; canonical-run and pinned-state checks

The verification pass at `06a3f82` did not repeat work whose inputs are unchanged and already verified. Specifically, the contract-level results of the Mythos audit are carried forward unchanged on the strength of the source hash, and the Daybreak verification of every proof and every public object of run `20261029` is accepted; the verification pass

re-checked the public index hash and bound every local run file to it. What the verification pass ran itself: the test gate at `06a3f82`; the M01 and M02 proofs of concept against the new approval tool; decoding and hashing of the five sealed packages; the reconstruction of every expected runtime hash and address from the compiled artifact and the manifests; one pinned read per chain of the custody Safe, the claim addresses and the Reserve Safe; the quarantine state of the superseded runs; the roster against the configuration; the documentation changes; the repository hygiene script; and, through one adversarial reviewer with no chain access, the run lock, run record, quarantine journal and run binding that closed BC-I11 and BC-I12.

Test gate at `06a3f82`

Test	Result
`forge build` and `forge test --offline` (Forge 1.8.1, pinned solc 0.8.36)	30 passed, 0 failed, 2 fork suites skipped without fork inputs
`tools/test_tools.py`	57 passed, 0 failed (the 22 stale assertions reported by the Daybreak reviews of `5a8937e` were repaired in `c8ced27`)
`toolkit.tests.test_bridge_allocations`	14 passed, 0 failed
`tools/package.py verify runs/bridge-allocations-20261029`	revision 6; five packages current; every file matches its seal
`claims_run.bind` of the canonical run against the public index	bound; five contracts; 22 index entries matched
`scripts/check-public-package.py`	fails on four tracked or untracked PDFs that contain NUL bytes (two in `vault/audit`, two audit outputs in `output/pdf`); no text file flagged

Proofs of concept re-run

Case (second-pass PoC)	Result at `06a3f82`
M01: every preflight check failing, signable mode	`SystemExit`, nothing but `claims.csv` and `manifest.json` in the label directory, run record empty
M01: no chain reads, signable mode	refused: "signable files need the chain checks (--rpc)"
M01: `--nonce 99` against Safe nonce 7	refused: "nonce is the Safe's"
M01: `--draft` without chain reads	writes only `approval-draft/`, listed as a draft in the run record
M02: EOA as `deployment.address`	refused before any chain read: "manifest address … is not the CREATE2 address …"

Case (second-pass PoC)	Result at `06a3f82`
M02: getter-faking contract at the real address	refused: "contract code is this BridgeClaims" check fails
M02: corrupted `init_code_keccak256`	refused: "init code hash differs"
Control: every check passing	`approval/` written, sealed and listed

Canonical run and pinned state

The public `SHA256SUMS` of run `20261029` hashes to the value in the audit identity table and lists 22 objects; every one of them matches the local run byte for byte. Rows per chain: Ethereum 187,005; BSC 620,276; Arbitrum 27,839; Base 245; Linea 1,389. The Merkle roots of all five chains are unchanged from the audited run on BSC, Arbitrum, Base and Linea; Ethereum's root changed with its list. For every manifest the verification pass rebuilt the init code and runtime code from the compiled artifact and the constructor arguments: every init-code hash and CREATE2 address equals the manifest, and every runtime hash equals the team's table and the approval packages' `spender_binding`.

Chain	Pinned block	Custody Safe	Claim address code	Other
Ethereum	26,297,003	4 roster owners, threshold 2, nonce 7	none	Reserve Safe nonce 61
BSC	130,436,009	4 roster owners, threshold 2, nonce 1	none	
Arbitrum	519,297,421	4 roster owners, threshold 2, nonce 1	none	
Base	53,235,409	4 roster owners, threshold 2, nonce 1	none	
Linea	32,460,722	4 roster owners, threshold 2, nonce 1	none	

Block hashes: Ethereum `0x91912ed4…b34ac`, BSC `0xb94769a7…82c05`, Arbitrum `0x17211753…bf3cb`, Base `0xefc9175d…b2812`, Linea `0x75ee8eb6…532df`. The owner sets equal the committed `custody-roster.json` on every chain; `claims_run.custody_requirements` accepts the bridge configuration against that roster.

The five packages: each `PACKAGE.sha256` verifies, and the seal hashes equal the run record. The four approval packages decode, with the verification pass's own Transaction Builder encoder and MultiSend packing, to one `approve(spender, total)` per asset (BSC additionally one WBNB `deposit()`), with the spender equal to the `20261029` manifest address and every amount equal to the manifest approval; the revoke files approve zero once per token; all record 58, 22, 19 and 19 passed checks and no failure, a pinned block and hash, and the manifest SHA-256. The Safe transaction hashes recomputed at nonce 1 equal the recorded ones and the Daybreak regrade's table: BSC `0x2e3ff92a…06950`, Arbitrum `0x27bd7ddf…268ea`, Base `0xf142bc2d…f12c8`, Linea `0xc5038a7f…dde99`. The release package holds one transaction: the Reserve Safe transfers exactly the Ethereum ONE approval (62,177,050.4682 ONE, the claimable total after the two held rows) to the custody Safe at nonce 61; `release-plan.json` records the five pinned blocks, the custody-roster hash, every manifest hash and the 22-entry published index, no escrow batch and 50 escrows skipped as empty.

Superseded runs: `20261018` and `20261028` hold no signable-looking file outside `quarantine/`, each has a `SUPERSEDED.md`, and `verify` finds nothing current in either.

Carried forward from the Mythos audit of `4840fa9`

These results depend only on the contract source, which is unchanged, and are not repeated:

- Baseline Forge suite 29 passed; 101 added tests for the claim, native and token paths (83 local, 9 Ethereum fork, 9 BSC fork), 56 for build, coverage gaps and gas, 10 for the payee failure-mode table; coverage of `BridgeClaims.sol` 100% on every metric with the gap tests.
- Stateful invariant campaign of 16,384 and 131,072 randomized calls with zero violations of I1, I3, I4 and I5; differential Merkle test of 1,366,030 cases against OpenZeppelin with zero disagreements.
- Bytecode reproducible across Forge 1.8.1, 1.8.6, 1.6.0-nightly, a clean clone and a direct `solc` run; `PUSH0`, `MCOPY` and transient storage execute on all five real nodes; the per-chain runtime derived on each real node.
- Independent verification of all 836,756 proofs of the audited run in Go and Node; 141 of 141 sampled rows paid exactly on pinned forks of all five chains with the real tokens; all 49 approval tokens identified, 15 upgradeable proxies and 9 with pause or blacklist controls, none active against the Safe.
- Escrow release complete and verified on chain (49 escrows empty, settings restored); the payee command executed end to end on a fork; the correction procedure rehearsed with real small-asset lists.

Crosswalk and status

Merged IDs are the team's. "Mythos" and "Daybreak" give each series' ID and rating at `4840fa9`; the merged rating is the one this report stands behind for `4840fa9`, with its reason; the status is at `06a3f82`.

Merged ID	Mythos (4840fa9)	Daybreak (4840fa9)	Merged rating and reason	Fix commits	Status at 06a3f82
BC-H01	BC-M02, Medium	BC-H01, High	Medium: the produced approvals named the manifest's spender and the address reproduced; the gap bit only with a manifest altered on the operator host, so it is defence in depth, not a wrong spender	`985980a`, `add3fe1`, `dee75f5`, `7941b81`	Verified remediated (Daybreak `91f408b` to `5a8937e`; PoC re-run here)
BC-H02	BC-L01, Low	BC-H02, High	Low: eight of the ten payees are key-controlled validator accounts paid by documented policy; the two contract rows were a per-chain policy inconsistency, not a loss through contract or tooling	`d0a3606`, `d495b5a`, `27125e9`; runs `20261028`, `20261029`	Verified remediated and published (two rows held; Daybreak `91f408b` rebuild and `5a8937e` bundle; counts re-checked here)
BC-M01	BC-L02, Low	BC-M01, Medium	Low: the address reproduces by four independent routes, the script is fail-safe under a real broadcast and stable Forge passes; an unpinned toolchain dependency, not a blocked chain	`dee75f5`	Verified remediated (direct factory call; Daybreak dry-run records; diff read here)
BC-M02	BC-M01, Medium	BC-M02, Medium	Medium: a process gap that could lead to signing stale hashes; both series agree	`add3fe1`, `626bd1e`, `b871754`, `9be0414`, `d0cceed`, `23eb632`, `9afc68d`, `87380e0`, `c8ced27`	Verified remediated in three rounds (Daybreak regrade; PoC re-run and package review here)
BC-M03	BC-L03, Low	BC-M03, Medium	Low: the 49-escrow release had already executed to the intended Safe and only the ONE transfer remained, gated by the fork replay; a defect in a tool with one remaining use	`24b7ca7`, `a4a580a`, `c8ced27`	Verified remediated (recipient from configuration, pinned reads, run and index binding)
BC-L04	BC-L04, Low	BC-L04 (from `5a8937e`), Low	Low: stale ONE stage-1 batch	release package `20261029` at nonce 61	Rebuilt and verified here; execution pending
BC-L05	BC-L05, Low	BC-L05, Low	Low	`9df98e2`, `0510737`	Code remediated: live nonzero allowances are reset first; CRV and AAG listed
BC-L06	BC-L06, Low	BC-L01 then BC-L06, Low	Low	`approve_claims.py` preflight, `0510737`	Code remediated: pinned `symbol()` shown beside the label, decimals drift rejected

Merged ID	Mythos (4840fa9)	Daybreak (4840fa9)	Merged rating and reason	Fix commits	Status at 06a3f82
BC-L07	BC-L07 (B1-L01), Low	BC-L07, Low	Low	`99786c3`; run `20261029`	Artifact remediated: every display amount equals the raw amount; roots unchanged
BC-L08	BC-L08, Low	BC-L08, Low	Low: design property	`61dd63a`, `d7e30c8`	Accepted and documented for relayers
BC-I01	BC-I01	BC-I01	Informational	none	Unchanged: no source-level trigger for the 0.8.36 advisory
BC-I02	BC-I02	BC-I02	Informational	none	Unchanged: issuer and proxy trust assumption
BC-I03	BC-I03	-	Informational	`c8ced27` (index binding)	Partly addressed: manifests now bind to the summary and the published index; `bytecode_keccak256` keeps its name
BC-I04	BC-I04	-	Informational	`61dd63a`, `47207b8`	Addressed in `DESIGN.md` and `RELEASE.md`
BC-I05	BC-I05	-	Informational	`495934f`, `a7f9303`	Partly addressed: `--chain` in the claim command, provider-key pattern; hygiene gate still red on PDFs; parsing permissiveness unchanged
BC-I06	BC-I06	-	Informational	none	Unchanged: contract errors and events
BC-I07	BC-I07	-	Informational	`61dd63a`	Addressed: tree construction and correction pitfalls documented
BC-I08	BC-I08	-	Informational	`d7e30c8`	Addressed: explorer verification settings in the runbook
BC-I09	BC-I09	BC-I09 (from `5a8937e`)	Informational	none	Open policy decision before opening

Merged ID	Mythos (4840fa9)	Daybreak (4840fa9)	Merged rating and reason	Fix commits	Status at 06a3f82
BC-I10	BC-I10	-	Informational	`87380e0`, `d7e30c8`	Partly addressed: roster committed, owner confirmation step and device limits in the runbook; key custody undocumented
BC-I11	-	BC-I11 (regrade of `5a8937e`)	Informational	`c8ced27`, `47207b8`, `82bbb26`	Closed: one run lock, run record, journaled quarantine; adversarial probes P1 to P4, P8, P9 all fail closed
BC-L09	-	- (new, verification pass)	Low: an operator deleting the lock file mid-refresh can cost a good package; fails closed	none	Open; recommendation given
BC-I12	-	BC-I12 (regrade of `5a8937e`)	Informational	`c8ced27`, `82bbb26`	Closed: `claims_run.bind` refuses every manifest mutation, bad label and index mismatch before any chain read; probes P5 to P7

Findings

Each entry gives the defect as found at `4840fa9`, both series' ratings where they differ, the merged rating, the fix and its verification, and the status at `06a3f82`.

BC-H01 - Approval and deployment tooling did not bind the spender to the CREATE2 commitment

Ratings: Daybreak High; Mythos Medium; merged Medium
Affected invariants: I6, I9, I10
Locations at 4840fa9: `tools/approve_claims.py:227-245,277,305-311`; `script/Deploy.s.sol:42-63`
Status at 06a3f82: Verified remediated

At `4840fa9` the spender in every `approve` call was `manifest.deployment.address`, copied verbatim; the tool never read `init_code_keccak256`, never compared the manifest's deployer and salt with `chains.json`, accepted an EOA or a getter-faking contract, and performed no check at all without `--rpc`. `Deploy.s.sol` bound the init code on its fresh-deploy path but accepted any code on its already-deployed path. The built manifests were self-consistent, so the defect bit only if a manifest was altered on the operator host after the build. The Mythos verifier confirmed the reproduction and the Medium rating; the Daybreak first pass rated it High under the wrong-spender rubric. The merged rating is Medium because the produced approvals named the manifest's spender and the address reproduced, which leaves a defence-in-depth gap rather than a wrong spender.

Fix (`985980a`, `add3fe1`, `dee75f5`, `7941b81`):

- `claim_tree.runtime_code` reconstructs the runtime code with immutables from the compiled artifact.
- `approve_claims.bound_deployment` recomputes the EIP-1014 address from the compiled creation code, the constructor arguments, the canonical deployer and the `chains.json` salt, and stops before writing when the deployer, salt, chain id, funder, wrapped token, root, count, list hash, constructor encoding, creation-code hash, init-code hash or address disagree.
- With `--rpc` it also checks the deployer's code hash and, once deployed, `extcodehash` against the exact runtime and the six getters.
- `Deploy.s.sol` recomputes `expected` with `vm.computeCreate2Address`, checks the deployer's code hash and the creation and init-code hashes, deploys with a direct call to the deployer, and on the already-deployed path compares the code hash with a local never-broadcast copy.

Verification:

- Daybreak `91f408b` verified the binding with negative tests (EOA spender, other factory in the manifest or in `chains.json`, other creation code, other list, other salt, getter-compatible contract) and kept it verified at `284dc77` and `5a8937e`.
- The verification pass re-ran the original PoCs: an EOA spender, a getter-faking contract and a corrupted init-code hash each stop before any file is written (table above).
- The verification pass rebuilt every expected runtime hash and address from the artifact and the `20261029` manifests and found them equal to the manifests, the team's table and the approval packages.

BC-H02 - Payees with code on Harmony and no code at the destination

Ratings: Daybreak High; Mythos Low; merged Low
Affected invariants: I3, I8
Locations at 4840fa9: `toolkit/scripts/bridge/build-bridge-allocations.py:86,167,197`; frozen Harmony classification at block 93,623,067
Status at 06a3f82: Verified remediated and published

Both series reproduced the counts: six Ethereum payees (six rows) and four BSC payees (five rows) carried code on Harmony and had no code at their destination. Eight of the ten are Harmony validator accounts, which store the staking record in the code field and are ordinary key-controlled accounts that upstream policy deliberately pays at the same address; one had already transacted on BSC and Ethereum. The remaining two are genuine Harmony contracts whose two Ethereum ONE rows came from holdings on BSC and Ethereum, where the pipeline classified the address per holder chain, while the same addresses' Harmony holdings were held. The Mythos verifier rated this Low: a policy inconsistency affecting two rows, not a loss through contract or tooling; the Daybreak first pass rated it High as an unclosed control requirement for ten payees. The merged rating is Low.

Fix (`d0a3606`, `d495b5a`, `27125e9`): `contract_elsewhere` holds an account without code where it holds whenever another classified chain has a contract at that address, with a reason, until a reviewed override decides it; a test covers it. The allocation run was rebuilt with the same inputs (`allocations-20261028`): exactly three allocation rows changed to `held_contract`, the two Ethereum claim rows disappeared, and the other four chains' claim inputs are byte-identical.

Verification:

- Daybreak `91f408b` independently rebuilt the allocations (5,032,539 verifier checks, zero failures) and verified all 836,754 leaves and proofs of the rebuilt package in Go.
- Daybreak `284dc77` and `5a8937e` downloaded the public bundles `20261028` and `20261029` anonymously and verified every indexed object, every proof and every display amount.
- The verification pass confirmed 187,005 Ethereum rows and 836,754 rows in the canonical run, that the public index hashes to the committed value and binds to every local file, and that the toolkit test passes.
- The Harmony archive node was unavailable to every pass, so the validator classification rests on the frozen artifact and Harmony's source behaviour; present custody of historical validator keys is not proven.

BC-M01 - Deployment depended on Forge's CREATE2 rewrite

Ratings: Daybreak Medium; Mythos Low; merged Low
Affected invariants: I6, I9
Locations at 4840fa9: `script/Deploy.s.sol:51-54`; `docs/RUNBOOK.md:24-29,65-70`
Status at 06a3f82: Verified remediated

`new BridgeClaims{salt}` inside a broadcast relied on Foundry rewriting the creation into a call to the deterministic-deployment proxy. Forge 1.8.1 does not apply the rewrite on its Optimism execution family (chain ids 10 and 8453, or `optimism = true`), so the Base dry run landed on a sender-derived address and the script's own check reverted; 1.8.6 passes on the live Base endpoint only because it has a native Base family; the nightly is correct everywhere. The manifest address was correct by four routes, the script sent zero transactions under a real broadcast, and no artifact or hash depended on the toolchain, so the Mythos verifier rated it Low; the Daybreak first pass rated it Medium as an operational blocker. The merged rating is Low.

Fix (`dee75f5`): the script calls the proxy directly with `salt ++ initCode` inside the broadcast, requires the returned 20 bytes to equal the recomputed address, checks the deployer's code hash, and prints the runtime code hash to compare with the signing sheet.

Verification: the team's dry runs under Forge 1.6.0-nightly and 1.8.1 land on the manifest address on all five chains (`runs/bridge-allocations-20261029/deploy-dry-run-20261029T055945Z/`, ten records with the expected runtime hashes); Daybreak `284dc77`, `5a8937e` and the regrade inspected those records and rebuilt the expected runtimes offline. The verification pass read the diff and reconstructed the same runtime hashes; it did not run the script. Each actual deployment still needs a fresh simulation on its target chain and the post-deployment check.

BC-M02 - Approval preflight was advisory, unpinned and self-referential

Ratings: Daybreak Medium; Mythos Medium; merged Medium
Affected invariants: I4, I10, I11
Locations at 4840fa9: `tools/approve_claims.py:174-184,190-270,284-294,350-356,459-467,475-481`
Status at 06a3f82: Verified remediated

At `4840fa9` failed checks only produced a `not yet` line: the four signable files were written unconditionally and the tool exited 0; reads used the `latest` block tag; `--nonce` silently replaced the chain nonce; the expected owner set was whatever the Ethereum Safe returned; the `MultiSendCallOnly` code was never read. All ten stored sheets were one nonce behind the live Safes. Both series rated it Medium under the stale-hash rubric.

Fix, three rounds:

- `add3fe1`, `626bd1e`: signable files need `--rpc` and every check at one pinned block (Safe identity, roster and threshold, nonce, `MultiSendCallOnly` and deployer code hashes, deployed runtime and getters, funding, final block-hash recheck); `--draft` writes only draft-named files marked "do not sign".
- `9be0414`, `b871754`, `d0cceed`, `23eb632`: approvals are built in an empty staging directory and published as a sealed package per contract; every contract of a run must pass before any is published; a failed refresh publishes nothing and moves the earlier package to `superseded/` with `INVALIDATED.md`; legacy signable files stop the tool until quarantined.
- `9afc68d`, `87380e0`, `c8ced27`: the owner roster is committed in `custody-roster.json` and a configuration naming other owners stops both tools; every refresh and quarantine holds one lock on the run; `PACKAGES.json` lists each current package with its seal and generation and is written first; `tools/package.py verify` refuses anything not listed; quarantine is journaled and recoverable.

Verification: Daybreak `91f408b` and `284dc77` found residuals (stale files surviving a failed refresh; seal and activation failures leaving mixed packages); Daybreak `5a8937e` found the earlier finalization PoC closed and narrower lock-scope and quarantine residuals; the Daybreak regrade verified the finding remediated and moved the residuals to BC-I11. The verification pass re-ran the M01 PoCs against `06a3f82` (table above), decoded the four current approval packages and recomputed their Safe hashes, confirmed the roster against the configuration and against the live owner sets, and confirmed that the superseded runs hold no signable file outside quarantine. The run-lock, run-record and quarantine changes of `c8ced27` were reviewed adversarially; see BC-I11.

BC-M03 - Release planning accepted an arbitrary custody recipient

Ratings: Daybreak Medium; Mythos Low; merged Low
Affected invariants: I8, I10, I11
Locations at 4840fa9: `tools/release_plan.py:141-150,364-371,401-406,437-442,475-487,964-965,979`
Status at 06a3f82: Verified remediated

The recipient of every escrow release and of the ONE funding was the `--safe` argument, defaulting to a hard-coded address and never checked against the configured custody multisig or the manifests' funder, and the owner roster used for the custody checks came from the Ethereum Safe. The 49-escrow release had already executed to the intended Safe, so only the ONE stage-1 transfer remained, gated by the fork replay; the Mythos verifier rated the remaining

exposure Low. The Daybreak first pass rated it Medium; its later reviews kept Medium for the incomplete manifest binding and unpinned reads until the regrade verified it. The merged rating at `4840fa9` is Low.

Fix (`24b7ca7`, `a4a580a`, `c8ced27`): the recipient is derived from the bridge configuration's `custody_multisig`, which must be the same on every chain of the plan and equal the ONE custody, and an explicit `--safe` must match it; the custody Safe on every chain must pass the Safe 1.3.0 identity checks and be owned by the committed roster; `--claims-dir` is required and `claims_run.bind` checks, before any chain read, that every summary entry and its manifest agree on label, chain, chain id, funder, root, row count, list hash, address and status, that each ready contract's list and tree hash to the manifest, that labels are unique plain names, and that every file of the run's published `SHA256SUMS` hashes the same locally; every chain read goes through a `PinnedRpc` whose block hash is read again before publishing; the release is built and published as a sealed package.

Verification: Daybreak `284dc77` and `5a8937e` confirmed the recipient derivation and the pinned reads and found the summary-to-manifest gap; the regrade verified the finding remediated and moved the gap to BC-I12. The verification pass bound the canonical run to the public index with `claims_run.bind`, decoded the release package (one transfer of exactly the Ethereum ONE approval to the custody Safe at Reserve nonce 61, five pinned blocks, every manifest hash, the roster hash and the 22-entry index recorded), and read the Reserve nonce live. The binding code was reviewed adversarially; see BC-I12.

BC-L04 - The pending ONE stage-1 batch

Rating: Low (both series)
Affected invariants: I8, I10
Status at 06a3f82: Rebuilt and verified; execution pending

The stored batch of run `20261018` was built for Reserve Safe nonce 58 while the Reserve Safe stood at 61 after four unrelated transfers. The release package of run `20261029` (generation `20261029T072103Z`) holds one transfer of 62,177,050.4682 ONE, exactly the Ethereum ONE approval of the canonical run, from the Reserve Safe to the custody Safe at nonce 61. At the verification pass's pinned Ethereum block the Reserve nonce was still 61 and the custody Safe held no ONE, so the package is current and the Ethereum ONE approval is the only unfunded one. The team's fork replay funds the ONE approval exactly. Rebuild the package if the Reserve nonce moves before signing, and have the signers compare the Safe transaction hash with the sheet.

BC-L05 - The approve-reset list omitted CRV and AAG

Rating: Low (both series)
Status at 06a3f82: Code remediated

CRV and AAG, like USDT, reject a nonzero-to-nonzero `approve`, so a re-approval batch built while their allowances were nonzero would revert as a whole. `9df98e2` makes signable mode read every allowance at the pinned block and insert `approve(0)` before any nonzero approval whenever the live allowance is nonzero; `chains.json` lists USDT, CRV and AAG for draft batches built without chain reads; `0510737` tests it. Daybreak `5a8937e` ran the test in isolation; the verification pass ran the full suite. Every current approval package was built from zero allowances and contains no reset beyond USDT's.

BC-L06 - Signing labels differed from live token symbols

Rating: Low (Daybreak BC-L01, then BC-L06; Mythos BC-L06)
Status at 06a3f82: Code remediated

The signable tool now reads `decimals()` and `symbol()` at the pinned block, fails on a decimals mismatch, and shows a differing on-chain symbol beside the bridge label on the sheet; the three known differences (Arbitrum USDT0 with the tugrik sign, Base MOODENG, BSC Cake) appear there. Token address and checked decimals remain the signing identity.

BC-L07 - The published human-readable amount column was wrong for 992 rows

Rating: Low (both series)
Status at 06a3f82: Artifact remediated

`format_amount` printed 86,167 cells in scientific notation and 992 of them 10^9 too large. `99786c3` formats with integer unit conversion; the lists were rebuilt into run `20261029`. Daybreak `5a8937e` compared the runs row by row: only `amount` changed, in 65,837 Ethereum, 20,298 BSC and 32 Arbitrum cells, every new value equals `amount_raw` at the token's decimals, trees and roots are unchanged, and the list hashes, addresses and runtime hashes changed on those three chains. The verification pass confirmed the resulting addresses and hashes from the artifact.

BC-L08 - A hostile native recipient forces about 3.6 million gas for its own row

Rating: Low (both series)
Status at 06a3f82: Accepted and documented

`_sendNative` forwards all gas; a recipient that burns it leaves 1/64 for the wrapped fallback, so the row needs about 3.55 million call gas (about 4 million as a transaction) and a cheaper attempt reverts with nothing moved. The cost falls on the submitter. `DESIGN.md` and `RUNBOOK.md` now tell relayers to estimate gas per row, cap it and skip rows that revert.

BC-L09 - The run lock is tied to a deletable file

Rating: Low (new finding of the verification pass)
Affected invariants: I10
Location at 06a3f82: `tools/package.py:199-215` (`run_lock`), `232-254` (`fail_closed` writes the in-memory record), `276-278` (cleanup of staging directories)
Status at 06a3f82: Open

`run_lock` takes an exclusive `flock` on `<run>/.packages.lock`, a plain file. If that file is removed while refresh A holds it (for example by a `git clean -x`-style cleanup of the gitignored `runs/` tree during a refresh), refresh B of the same target creates a new lock file, completes, and is listed; when A then fails, its failure handler moves B's package to `superseded/` with `INVALIDATED.md` and writes the record from A's memory, which no longer lists B's package. Every interleaving observed fails closed: nothing signable results, and `verify` reports the run accordingly. The cost is a good package and operator confusion, which is griefing under the plan's rubric.

Recommendation: take the lock on a descriptor of the run directory itself, or after locking compare the descriptor's inode with the path's and refuse when they differ; have `fail_closed` re-read `PACKAGES.json` under the lock instead of writing the in-memory copy; remove leftover staging directories only when their pending entry predates the current lock holder.

Informational observations

BC-I01 - Solidity 0.8.36 advisory

Unchanged: the source and compiler configuration are unchanged, only `SOL-2026-7` can apply, and its trigger pattern is absent from the generated utility Yul and both assembly blocks. A compiler upgrade would change every address and artifact.

BC-I02 - Token success remains an issuer and proxy trust assumption

Unchanged: of the 47 listed ERC-20s, 46 return a 32-byte true and USDT returns nothing; 15 are upgradeable proxies and 9 have pause, blacklist or freeze controls, none active against the Safe at the audited pins; PAXG charged no fee. Per-asset canaries with balance-delta monitoring and a tested revoke path remain mandatory at opening.

BC-I03 - Manifest binding and naming

`claims_run.bind` now ties every manifest to the summary and to the published index before any chain read, which closes the anchoring gap the Mythos audit noted for `summary.json`. The manifest field `bytecode_keccak256` keeps its name (it holds the creation-code hash) because renaming it would change every manifest; `Deploy.s.sol` still does not cross-check that a manifest with native rows names a wrapped token, which the builder refuses and a claim would reject with `NativeNotSupported`.

BC-I04 - Documentation

Addressed in `61dd63a` and `47207b8`: gas figures, the PAXG fee statement, the reset rule, the correction pitfalls, the ONE token identity and the handoff gas statements. The NatSpec comment on the bitmap position is left so the audited source stays byte-identical.

BC-I05 - Tooling gaps

Partly addressed: `proof.py` pins its claim command with `--chain` (`495934f`); `check-public-package.py` rejects provider keys in URLs (`a7f9303`); the tool test suite is green at `06a3f82`. The whole-repository hygiene gate is still red because four PDFs contain NUL bytes, so it cannot serve as a clean publication gate until binary files are excluded or allowed explicitly. The builder's permissive address and amount parsing is unchanged; the published inputs are canonical and hash-anchored.

BC-I06 - Error and event surfaces

Unchanged: `TransferFailed` is overloaded across allowance, balance, blacklist, pause and gas starvation, and no event names the submitter. Contract-level; documented for payee support.

BC-I07 - Claim semantics

Addressed: `DESIGN.md` and the public README state that OpenZeppelin's `StandardMerkleTree.verify` accepts the proofs while `StandardMerkleTree.of()` builds a different tree, and `DESIGN.md` gives the two correction pitfalls (exclude by account and token; read `claimedBitmap` after the revoke is mined).

BC-I08 - Explorer verification

Addressed: `RUNBOOK.md` §5 gives the exact standard-JSON settings and the Sourcify limitation of a metadata-free build.

BC-I09 - Economics of the payee set

Open policy decision: about 249,000 rows are worth less than the gas to claim them; the artifacts contain no minimum and no relayer plan. Decide, before opening, whether a Safe-funded relayer pays dust rows with `claimMany` or a minimum applies.

BC-I10 - Key custody and signing

Partly addressed: the four-owner roster is committed (`87380e0`), the tools refuse a configuration that disagrees with it, the runbook requires each owner to confirm their own address before the first signature and describes what hardware wallets can and cannot show. Who holds the keys, on which devices and in which jurisdictions is still not documented.

BC-I11 - Package coordination and failure reporting (Daybreak regrade)

The Daybreak regrade recorded four residuals after `5a8937e`: single-label and all-label refreshes took different locks; quarantine took no lock and could stop part-way with no record; a package that could not be moved stayed current while the tool printed `COULD NOT CLEAR`; a symlinked label directory was accepted; and no run-wide marker let a signer reject an interrupted activation. `c8ced27` closes them: every approval refresh, release refresh and quarantine locks `<run>/.packages.lock` on the run directory resolved once; `PACKAGES.json` lists each current package with the hash of its seal and its generation, is written first and marked pending until every target is in place, and `tools/package.py verify` refuses anything not listed; quarantine writes `QUARANTINE.json` before moving anything, puts everything back on a failed move, and leaves the record pending until `--recover` completes an interrupted one; every path component from the run directory down is checked for symlinks.

The adversarial review of the verification pass (one reviewer, scratch copies with synthetic runs, no chain access) ran ten probes with 104 checks:

- concurrent refreshes (single-label against all-label, two single-label, the same run through a symlinked path) and a concurrent quarantine are refused by the one lock;
- a process killed between the first and second target rename leaves nothing current and `verify` exits 1; a later refresh recovers;
- a real permission denial on `superseded/` leaves the unmovable package unlisted, so it is not current;
- symlinked label directories, run-root components, record, lock, `superseded/` and `quarantine/` entries are refused by `staged`, `bind` and `verify`;
- a quarantine killed after its first move blocks refreshes and new quarantines until `--recover` completes it; a failed recover leaves the run blocked and nothing current;
- verify path aliases and package-like directories planted under `superseded/` cannot make an unlisted package current.

Status: closed. One new residual is recorded as BC-L09, and three hygiene notes remain informational: `release_plan.py` binds the run and fetches the published index inside `package.staged`, so a network failure during a release rebuild invalidates the current release package (fail-closed, but it costs a good package; bind first, as the approval tool does, and prefer `--published-index` with a local copy on signing days); `claims_run.LABEL` accepts the reserved names `release`, `superseded` and `quarantine` (listed packages are still verified by exact path and the summary is bound to the index, so no exploit path was found; reject them in `bind`); a crash between writing and renaming the record

leaves a dot-prefixed `.PACKAGES.json.*.partial`, the index duplicate check runs before whitespace stripping, and index entries may resolve through an in-run symlink (remove stale partials under the lock, strip first, and refuse symlink components as `package.run_path` does).

BC-I12 - Claim-run consistency checks (Daybreak regrade)

The Daybreak regrade recorded that `claim_run()` accepted a manifest whose root, chain id, claim count, list hash, address and `claims.csv` differed from an unchanged `summary.json`, and that explicit approval `--labels` did not require a ready summary entry. `c8ced27` adds `tools/claims_run.py`: before any chain read, `bind` requires every summary entry and its manifest to agree on label, chain, chain id, funder, root, row count, list hash, address and status, each ready contract's `claims.csv` and `tree.txt` to hash to the manifest's digests, labels to be unique plain names with no manifest in the run outside the summary, every explicit label to be a ready entry named once, and, for signable approvals and every release, every file of the run's published `SHA256SUMS` to hash the same locally. Both tools call it; the release plan records the comparison.

The verification pass bound the canonical run to the public index (five contracts, 22 entries matched), and the adversarial pass confirmed the refusals: every manifest mutation (root, chain id as another value and as a string, claim count, list hash, address, `claims.csv`, `tree.txt`, all at once) against an unchanged summary is refused by `bind` and by the real `approve_claims.py --draft` command before the record is touched; non-ready, unknown, duplicated, path-like, trailing-slash, differently cased, reserved and empty labels are refused; a changed digest, a missing file, an omitted required file, out-of-run entries and duplicates in the index are refused; manifest data is held in memory after `bind` and `claims.csv` is re-hashed by the spender binding, so there is no time-of-check gap before the chain reads.

Status: closed. The team's test `test_release_rejects_a_manifest_the_summary_does_not_commit` reproduces the regrade's case and names all five fields.

Invariant verdicts

ID	Verdict at 06a3f82	Basis and remaining gate
I1	Pass	Contract unchanged; bitmap, rollback, batch, reentrancy and invariant-campaign evidence carries forward.
I2	Pass for the published package	Every proof of run `20261029` verified independently (Daybreak `5a8937e`); the public index verified and bound locally here.
I3	Pass for the published package	The leaf binds the account; no row pays a system address; the two Harmony-contract rows are held; validator accounts paid by policy.
I4	Pass at code level; live allowance pending	Movement is allowance-bounded; every approval equals its asset's row sum; the spender is bound to the bytecode; allowances are zero until the packages execute.
I5	Pass	The contract ends every path with zero coin and zero wrapped balance; sweep pays only the funder.
I6	Pass at code and dry-run level	Direct factory call, init-code and runtime binding, reproducible addresses; a fresh simulation and post-deployment check remain per chain.
I7	Pass at code level; execution pending	Every token accepts X to 0 to X; nonzero allowances are reset first; a revoke package exists at nonce 1 and must be refreshed if the nonce moves.
I8	Not established for production	ONE stage 1, deployment, allowances, canaries and the dust policy remain open.
I9	Pass for the published bytes	Lists, trees, manifests, init code, addresses, runtime hashes and the public and private indexes agree; post-deployment immutable checks remain.
I10	Pass for the current packages	Every call and hash of the five packages reproduces independently at the recorded nonces, which were still live at the verification pass's pins; packages must be refreshed if a nonce moves; BC-L09 can invalidate a good package but not produce a wrong one.
I11	Pass with a documentation gap	No party beyond the 2-of-4 Safe is needed; owner key custody is undocumented.

Per-chain production decision

Chain	Sign approvals	Deploy	Open claims
Ethereum	NO until ONE stage 1 executes and an Ethereum approval package is built; ONE stage 1 itself: conditional GO at nonce 61	Conditional GO - fresh target-chain simulation, then post-deployment check	NO - ONE funding, deployed code, allowance and canaries unverified
BSC	Conditional GO - package current at nonce 1; verify, owners confirm roster, refresh if the nonce moves	Conditional GO - as above	NO - deployed code, allowance and canaries unverified
Arbitrum	Conditional GO - as BSC	Conditional GO - as above	NO - as above
Base	Conditional GO - as BSC	Conditional GO - as above	NO - as above
Linea	Conditional GO - as BSC	Conditional GO - as above	NO - as above

Conditions for signing: `tools/package.py verify runs/bridge-allocations-20261029 <label>/approval` reports the package current; the Safe nonce equals the package's (or the package is refreshed with `--rpc --replace`); each owner has confirmed their address in `custody-roster.json`; a second party decodes every call and reproduces the Safe transaction hash; signers compare the domain and message hashes on their devices with the sheet. Conditions for deployment: deploy from the same run with the audited script, simulate on the target chain immediately before broadcasting, then verify the runtime hash and all six immutables on the real node and verify the source on the explorer. Conditions for opening: allowances equal the approvals; at least 100 random rows plus the last and one altered row verified through the real RPC; per-asset canaries succeed with a documented revoke process; the dust-row policy decided.

Deployment commitments

Chain	Contract address	Root	Init-code Keccak
Ethereum	0xcc0407394a4d7eb59f894abbb9c005631183d49a	0x964894effbc2fcaa5e87b9583ae6636959dd23fc1d4f2c3cd97db12d52f2523b	0xc2e0b6e1c3c316f37e53384093b6550eb07d04dc76150301e78c093cb70464cc
BSC	0xaffa52187721fb4165071349bbd8c55c8d4356fc	0x734794900e6066eac22a05de2031fa848fcf59ac6f8da10fac1de211bb9c81c4	0xa562128309af307bf4cab369a18ef1567f0426a0672ed3f04a315f6520183284
Arbitrum	0x2cf2afd2b8be95f9ba5e8a6b021e7895d6729469	0x39e92752231cebd64190cb5cf44e63ae75d38a1d42b63b85c3dbae48ae1037df	0x2d0a3eb67c27e37be26eaeb1f39ae8a6ecf53a6394c06c0630b3814ff4f0b3e0
Base	0xa2bddf296a5a945f0bb92e71629a3e78f401adbb	0x183e4b527d29eea5426e643c22ba0d21be7488d207db4cdfdd7278e68dcefe06	0x346d2cb3053c6039ad9ab5816c617457178668d9cbaf3aab2b2faa0df46498c7
Linea	0x8a5607026b92977c34aba988c5e22fc8b615ceaf	0xa891a83a107cba0d21c67e9038061be945b2c62e3af6b7bd2299456bb7902772	0x7c4b9982fb71fe9ebfcfcde607606508512585bd5da25e465825991fb70c5668

Expected runtime code hashes, rebuilt in the verification pass from the compiled artifact and the manifests and equal to the team's table and the approval packages: Ethereum `0xed9957a85f2ff3d089aa2f5cd760ce4afbdb1743b3d66770caeddfda26bdd2d3`, BSC `0xdb4763abb510a261308e26bd3a80e5fa1d32b6f9eabcb08e011858d4311ffc93`, Arbitrum `0xccde05f46873541165bd89c0c6fd06c2cb00a13b5ddb3423a5acbc1f198ba0b9`, Base `0xa301f9ed766c50c19b603208641a535e15d534c023a11e127b031a4825f33db7`, Linea `0x62b4fc5e0faff1ed42efd514607ac655ac09bdaadebdb29efda5719ff753301d`. The post-deployment check must compare `extcodehash` with these. No claim address had code at the verification pass's pinned blocks.

Artifact commitments

Public bundle `bridge-allocations-20261029`, served at the public bucket and verified in the verification pass against its `SHA256SUMS`:

Chain	Artifact	SHA-256
Ethereum	`manifest.json`	`09f57711875d598b3b1f334ea46e4323ee07858b75521bebe8a2189ad54b9fee`
Ethereum	`claims.csv`	`a932076291d645a03f9e67511ae64632c394c2d21afc9cc1093bbc7fe6af9707`
Ethereum	`tree.txt`	`39f3c0476bc1170809d0280dd01de78fbd710797ea3411486ed268609d413221`
BSC	`manifest.json`	`9284c27532202a576f9acb7e7d8ab581e78e583fa2941cc5771b3e789ce7fdd3`
BSC	`claims.csv`	`370c274681f08ab5b16d8bc697944e0a03d3ea35672e9aa793e9db5eaa768b8f`
BSC	`tree.txt`	`0f92c14d845a4f26df49528d31e01819f475c890628a3c52d840e2696ced4c72`

Chain	Artifact	SHA-256
Arbitrum	`manifest.json`	`e799b90745e97fb3803af342d3292772cfb059d7315e6a32a617500b4dce4861`
Arbitrum	`claims.csv`	`63bcab8c72823df0ef275f6cac7ae02e7a1c8ef8d937a0d2a78048ae4f54f0d1`
Arbitrum	`tree.txt`	`f5028f2f4bbd5a4fb157f005907edd2cc7ce76ff7925187a7242e510d444845f`
Base	`manifest.json`	`ce21f68a73f1bf332862f9b90562f02f560a2a3fdb9ac112c0332eb66dea9233`
Base	`claims.csv`	`63f1b1142df38f4531fdd8c5366628206684509518 34c21b76a8fff9e0dff1e0`
Base	`tree.txt`	`ed2872198d95bdc46508331bbd0da5595e8f1de1210d17ceafa43e854412a9e9`
Linea	`manifest.json`	`7179e94698f24201a530af677fd9fe2a46e13953df3f2b7f175ff411711ca0bf`
Linea	`claims.csv`	`30e030491a0e183cc9f70312b2c88b5a923ab3a21b044a07f613ef36c28f21af`
Linea	`tree.txt`	`6fa72956f7c6b7ef3be7f0304ad432a01df9b7deb6a9909e1568201a4bbc66ba`

Run-level commitments: `summary.json` `18917b4c1deebbf8e807667daea855c45f0000b1807027c40e8606a53d639fb5`; `CLAIM_CONTRACTS.md` `d38c4d1f4d7a6a28d800fcb35f8d7e0ff7b35eefcf94e1183208215256ffd732`; `release/release-plan.json` `d03af0c10f732442484015ced7ae0cfce46cc17816bb419fa556621441e63e53`.

Sealed packages current in `PACKAGES.json` revision 6. A seal identifies the bytes the verification pass decoded; it is not authorization to sign, which the per-chain decision governs.

Package	Generation	`PACKAGE.sha256` SHA-256	Safe transaction hash (nonce)
`bsc/approval`	`20261029T071448Z-41ce7d52`	`f31e1de55315317a8e08f23b631cab46b83308b74df8cd2464a535c595747d24`	`0x2e3ff92adb413103013702fe48e4fe2601faa3dd81013a55ebe8b591af206950` (1)
`arbitrum/approval`	`20261029T071448Z-41ce7d52`	`81d2e357e47ccac64daf4682b952e6ce2289535ef4febc447c80c06dd2f4de32`	`0x27bd7ddffc70425b0f989daeb3bb3e7d4d30da7860ba0d7dc2f9ddef9b0268ea` (1)
`base/approval`	`20261029T071448Z-41ce7d52`	`b0ae554d65a57ed968dd180676d08b3a48c9237a571ee73ccc12b944e1530aa8`	`0xf142bc2d2c1283c40dc3428f869ff2fd6b8b0036939361e07090e6f72d0f12c8` (1)
`linea/approval`	`20261029T071448Z-41ce7d52`	`0ee1bdf6f5a81ea09c4acc20e83927261abd6595a2880e7b4a71fd90988c9f4e`	`0xc5038a7f1b94d377b498317c122058a5e6f75dca09b22fe5a22c27c104fdde99` (1)
`release` (ONE stage 1)	`20261029T072103Z-d49f12bb`	`a05bd8a506c840de6b6d9dc027a42c5d1d9682fb2454b55ef39d793414cc53e9`	`0xcf8b0b50f1404013e94666c178d66426b54c654fcfa3bf277e3ed865b1018e78` (Reserve Safe, 61)

The revoke batches of the four approval packages hash, at nonce 1, to BSC `0x94adbfee…e9bd1`, Arbitrum `0xfd5749af…5996b`, Base `0xf7d6981b…6029ed` and Linea `0x56c09dd5…d9096f`; a revoke must be rebuilt at the live nonce when used.

Limitations and evidence boundary

- No production claim contract is deployed. Deployment simulation, runtime and immutable checks, allowances, canaries and explorer verification remain future gates; the verification pass did not run the deployment script and relies on the team's ten dry-run records, inspected by the Daybreak reviews, and on its own reconstruction of the expected runtime hashes.
- Chain state was read once per chain at the pinned blocks above. Nonces, balances, code and owners can change afterwards; every package must be verified current and refreshed if a nonce moves.
- Daybreak's verification of every proof and public object of run `20261029`, and its private-bucket checks, are accepted; the verification pass re-verified the public and private index hashes and bound every local run file to the public index, but did not re-verify every proof.
- The adversarial review of the run lock, run record, quarantine journal and run binding ran in a scratch copy with synthetic runs and no chain access.
- The Harmony archive node was unavailable to every pass; the validator-account classification rests on the frozen artifact and Harmony's source behaviour, and present custody of historical validator keys is not proven.
- Owner identities were compared only as address sets against the committed roster; intent is attested by the team's confirmation of 28 October and by each owner's confirmation step, not by the verification pass. Hardware-wallet display behaviour comes from vendor sources.
- The repository hygiene script fails on binary PDFs and was not a clean gate at `06a3f82`.
- Future proxy upgrades, issuer actions, fees, blacklists, pauses and token behaviour can change claimability after this report. Escrow release received integration-level review, not a full audit of every external LayerZero, Safe, token or escrow implementation.

Model and program disclosure

Mythos 5.1, Fable 5.1 Max, GPT-6 Astra Ultra, Daybreak Blue, and Opus 5.5 were used for preparing and conducting this audit. Model output was advisory; findings were accepted only when supported by code, tests, traces, pinned reads, or independently reproduced hashes.

hiddenstate.xyz participates in OpenAI's Trusted Access for Cyber (TAC) and Anthropic's Cyber Verification Program.

Appendix - reviewed source

Original source, 287 lines; SHA-256 94f8bed2e34d5a55a2304e96ab256cf59e980f5c813271c5659fd397c0f71255.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.36;

interface IERC20Minimal {
    function balanceOf(address account) external view returns (uint256);
    function allowance(address owner, address spender) external view returns (uint256);
}

interface IWrappedNative {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}

/// @title BridgeClaims
/// @notice Pays a fixed, pre-published list of bridge-collateral claims on one chain, every
            asset of
///         that chain in one list. Tokens move straight from the custody multisig to each payee.
///
/// How it works, in plain terms:
///
/// 1. Off-chain, every claim on this chain (one row per payee and asset) gets a number (`index`)
///    and is reduced to a 32-byte fingerprint (`leaf`). All fingerprints are combined into one
///    32-byte `root`. The list and the root are published so anyone can recompute them.
/// 2. This contract is deployed with that `root`, the custody multisig (`funder`), the wrapped
///    native coin and the chain id baked in. Nothing can be changed afterwards: there is no
            owner,
///    no admin, no pause and no upgrade. If the list ever needs to change, a new contract is
///    deployed with a new root.
/// 3. The funder approves this contract, per asset, for exactly the total of that asset's rows.
///    Tokens stay in the funder until claimed. The funder can stop the claims of any asset at any
///    time by lowering or revoking its approval; that needs nothing from this contract.
/// 4. A claim submits a row (index, account, token, amount) and the sibling hashes that connect
///    its fingerprint to `root`. The contract checks them, marks the row paid, and moves `amount`
///    of `token` from the funder to `account`. Anyone may submit a claim, but the tokens always
            go
///    to the row's `account`, and each row pays once.
/// 5. Native coin rows (ETH, BNB) use token address 0. The funder holds and approves the wrapped
///    coin (WETH, WBNB); the contract unwraps it and sends the coin. An account that does not
///    accept the coin receives the wrapped token instead.
///
/// The contract holds no tokens between transactions. Anything sent to it by mistake can be
/// returned to the funder by anyone with `sweep`.
contract BridgeClaims {
    // ------------------------------------------------------------------ fixed at deployment

    /// @notice Token address used in rows that pay the chain's native coin.
    address public constant NATIVE = address(0);

    /// @notice The custody multisig that holds the assets and approves this contract.
    address public immutable funder;
    /// @notice The wrapped native coin (WETH, WBNB) the funder approves for native rows; zero on
            a
    ///         chain without native rows.
    address public immutable wrappedNative;
    /// @notice The chain this list belongs to; deployment on any other chain reverts.
    uint256 public immutable chainId;
    /// @notice Fingerprint of the complete published list. Never changes.
    bytes32 public immutable root;
    /// @notice Number of rows in the published list. Indexes run from 0 to claimCount - 1.
```

```
    uint256 public immutable claimCount;
    /// @notice SHA-256 of the published list file, so the contract can be tied to the exact file
    ///         people download. Informational only; it is not used in any check.
    bytes32 public immutable listHash;

    // ---------------------------------------------------------------- state

    /// @dev Bit i of word i/256 is set once row i has been paid.
    mapping(uint256 => uint256) private _claimedBitmap;

    // ---------------------------------------------------------------- types, events and errors

    /// @notice One row of the published list together with its proof.
    struct Claim {
        uint256 index;
        address account;
        address token;
        uint256 amount;
        bytes32[] proof;
    }

    event Claimed(uint256 indexed index, address indexed account, address indexed token, uint256
            amount);
    /// @notice A native-coin row paid in the wrapped coin because the account refused the coin.
    event PaidWrapped(uint256 indexed index, address indexed account, uint256 amount);
    event Swept(address indexed token, uint256 amount);

    error ZeroAddress();
    error ZeroValue();
    error WrongChain(uint256 expected, uint256 actual);
    error IndexOutOfRange(uint256 index, uint256 claimCount);
    error AlreadyClaimed(uint256 index);
    error InvalidProof(uint256 index);
    error NativeNotSupported();
    error TransferFailed(address token, address to, uint256 amount);
    error UnexpectedNativeSender(address sender);

    // ---------------------------------------------------------------- construction

    /// @param funder_        the custody multisig holding the assets (approves this contract)
    /// @param wrappedNative_ the wrapped native coin, or zero if the list has no native rows
    /// @param chainId_       the chain the list belongs to
    /// @param root_          fingerprint of the published list (see `leaf`)
    /// @param claimCount_    number of rows in that list
    /// @param listHash_      SHA-256 of the published list file
    constructor(
        address funder_,
        address wrappedNative_,
        uint256 chainId_,
        bytes32 root_,
        uint256 claimCount_,
        bytes32 listHash_
    ) {
        if (funder_ == address(0)) revert ZeroAddress();
        if (root_ == bytes32(0) || claimCount_ == 0) revert ZeroValue();
        if (chainId_ != block.chainid) revert WrongChain(chainId_, block.chainid);
        funder = funder_;
        wrappedNative = wrappedNative_;
        chainId = chainId_;
        root = root_;
        claimCount = claimCount_;
        listHash = listHash_;
    }

    // ---------------------------------------------------------------- claiming
```

```
/// @notice Pay one row of the published list to its account.
/// @param index   position of the row in the published list
/// @param account the payee
/// @param token   the asset (an ERC-20 address, or `NATIVE` for the native coin)
/// @param amount  the amount in the asset's smallest unit
/// @param proof   the sibling hashes that connect this row's fingerprint to `root`
function claim(uint256 index, address account, address token, uint256 amount, bytes32[]
        calldata proof)
    external
{
    _claim(index, account, token, amount, proof);
}

/// @notice Pay several rows in one transaction, for example every asset of one payee. Rows
        that
///         were already paid are skipped, so a batch is not blocked by someone claiming one
        of its
///         rows first; any other failure reverts the whole batch.
/// @return paid the number of rows paid by this call
function claimMany(Claim[] calldata claims) external returns (uint256 paid) {
    for (uint256 i = 0; i < claims.length; ++i) {
        Claim calldata c = claims[i];
        if (c.index < claimCount && isClaimed(c.index)) continue;
        _claim(c.index, c.account, c.token, c.amount, c.proof);
        ++paid;
    }
}

// ------------------------------------------------------------------ read-only helpers

/// @notice The fingerprint of a row: keccak256 of keccak256 of the ABI encoding of
///         (index, account, token, amount). The double hash keeps row fingerprints distinct
        from
///         the internal nodes of the tree, following the OpenZeppelin convention.
function leaf(uint256 index, address account, address token, uint256 amount) public pure
        returns (bytes32) {
    return keccak256(bytes.concat(keccak256(abi.encode(index, account, token, amount))));
}

/// @notice True if a row belongs to the published list. Lets anyone check a row and its proof
///         without sending a transaction.
function verify(uint256 index, address account, address token, uint256 amount, bytes32[]
        calldata proof)
    external
    view
    returns (bool)
{
    return index < claimCount && _verify(proof, leaf(index, account, token, amount));
}

/// @notice True once row `index` has been paid.
function isClaimed(uint256 index) public view returns (bool) {
    return _claimedBitmap[index >> 8] & (1 << (index & 0xff)) != 0;
}

/// @notice Raw bitmap word; bit i of word w corresponds to row w * 256 + i.
function claimedBitmap(uint256 word) external view returns (uint256) {
    return _claimedBitmap[word];
}

/// @notice How much of an asset can be claimed right now: the smaller of the funder's
        approval to
///         this contract and the funder's balance. For `NATIVE` this is the wrapped coin.
function available(address token) external view returns (uint256) {
    address asset = token == NATIVE ? wrappedNative : token;
```

```
    if (asset == address(0)) return 0;
    uint256 approved = IERC20Minimal(asset).allowance(funder, address(this));
    uint256 balance = IERC20Minimal(asset).balanceOf(funder);
    return approved < balance ? approved : balance;
}

// ------------------------------------------------------------------ mistakes

/// @notice Return this contract's whole balance of `token` (or of the native coin, for
        `NATIVE`)
///         to the funder. The contract never needs to hold anything, so under normal
        operation
///         there is nothing to move. Anyone may call it; the destination is always the
        funder.
function sweep(address token) external {
    uint256 amount;
    if (token == NATIVE) {
        amount = address(this).balance;
        if (!_sendNative(funder, amount)) revert TransferFailed(NATIVE, funder, amount);
    } else {
        amount = IERC20Minimal(token).balanceOf(address(this));
        _safeTransfer(token, funder, amount);
    }
    emit Swept(token, amount);
}

/// @dev Only the wrapped coin sends the native coin here, while unwrapping a claim.
receive() external payable {
    if (msg.sender != wrappedNative) revert UnexpectedNativeSender(msg.sender);
}

// ------------------------------------------------------------------ internal

function _claim(uint256 index, address account, address token, uint256 amount, bytes32[]
        calldata proof)
    private
{
    if (index >= claimCount) revert IndexOutOfRange(index, claimCount);
    uint256 word = index >> 8;
    uint256 bit = 1 << (index & 0xff);
    uint256 bits = _claimedBitmap[word];
    if (bits & bit != 0) revert AlreadyClaimed(index);
    if (!_verify(proof, leaf(index, account, token, amount))) revert InvalidProof(index);
    _claimedBitmap[word] = bits | bit;
    emit Claimed(index, account, token, amount);

    if (token == NATIVE) {
        _payNative(index, account, amount);
    } else {
        _safeTransferFrom(token, funder, account, amount);
    }
}

function _payNative(uint256 index, address account, uint256 amount) private {
    address wrapped = wrappedNative;
    if (wrapped == address(0)) revert NativeNotSupported();
    _safeTransferFrom(wrapped, funder, address(this), amount);
    IWrappedNative(wrapped).withdraw(amount);
    if (!_sendNative(account, amount)) {
        IWrappedNative(wrapped).deposit{value: amount}();
        _safeTransfer(wrapped, account, amount);
        emit PaidWrapped(index, account, amount);
    }
}
```

```
    /// @dev Sends the coin without copying any return data, so a recipient cannot make the call
    ///      expensive by returning a large payload.
    function _sendNative(address to, uint256 amount) private returns (bool sent) {
        assembly ("memory-safe") {
            sent := call(gas(), to, amount, 0, 0, 0, 0)
        }
    }

    function _safeTransferFrom(address token, address from, address to, uint256 amount) private {
        if (!_tokenCall(token, abi.encodeWithSelector(0x23b872dd, from, to, amount))) {
            revert TransferFailed(token, to, amount);
        }
    }

    function _safeTransfer(address token, address to, uint256 amount) private {
        if (!_tokenCall(token, abi.encodeWithSelector(0xa9059cbb, to, amount))) {
            revert TransferFailed(token, to, amount);
        }
    }

    /// @dev Accepts tokens that return `true` and tokens that return nothing (such as Ethereum
            USDT);
    ///      rejects a `false` return, a revert, and an address without code.
    function _tokenCall(address token, bytes memory data) private returns (bool) {
        (bool ok, bytes memory ret) = token.call(data);
        if (!ok) return false;
        if (ret.length == 0) return token.code.length != 0;
        return ret.length >= 32 && abi.decode(ret, (uint256)) == 1;
    }

    /// @dev Standard sorted-pair Merkle proof check: hash the leaf with each sibling in turn,
    ///      smaller value first, and compare the result with `root`.
    function _verify(bytes32[] calldata proof, bytes32 leaf_) private view returns (bool) {
        bytes32 h = leaf_;
        for (uint256 i = 0; i < proof.length; ++i) {
            bytes32 p = proof[i];
            h = h < p ? _hashPair(h, p) : _hashPair(p, h);
        }
        return h == root;
    }

    function _hashPair(bytes32 a, bytes32 b) private pure returns (bytes32 out) {
        assembly ("memory-safe") {
            mstore(0x00, a)
            mstore(0x20, b)
            out := keccak256(0x00, 0x40)
        }
    }
}
```